- First, if `ftReserves < netOut + feeAmt` - unnecessary debt will be taken on the user (`_issueFt` takes new ft in gt).
- Second - `_issueFtToSelf` updates `transientStorage` at the end of execution, indicating the current balance of the contract - but this balance is not `balanceOf`, but `targetFtReserve` - that is, an invalid value.

```
function _issueFtToSelf(uint256 ftReserve, uint256 targetFtReserve, OrderConfig memory config) internal {
    if (config.gtId == 0) revert CantNotIssueFtWithoutGt();
    uint256 ftAmtToIssue = ((targetFtReserve - ftReserve) * Constants.DECIMAL_BASE)
        / (Constants.DECIMAL_BASE - market.issueFtFeeRatio());
    market.issueFtByExistedGt(address(this), (ftAmtToIssue).toUint128(), config.gtId);
    setTransientFtReserve(targetFtReserve);
}
```

An invalid value in tstore will further cause the invalid value to be passed to the callback - at least this is bad because this callback is used in Vault and Vault performs its calculations depending on `deltaFt`.

```
if (address(_orderConfig.swapTrigger) != address(0)) {
    int256 deltaFt = ft.balanceOf(address(this)).toInt256() - getTransientFtReserve().toInt256();
    int256 deltaXt = xt.balanceOf(address(this)).toInt256() - getTransientXtReserve().toInt256();
    _orderConfig.swapTrigger.swapCallback(deltaFt, deltaXt);
}
```

Proof of Concept: To see that the values are indeed incorrect, paste these two debug outputs into this location in the `_buyToken` function:

```
if (tokenOut == ft) {
    uint256 ftReserve = getTransientFtReserve();
    console.log('Reserves', ftReserve);
    console.log('Real Balance', ft.balanceOf(address(this)));
    console.log('HERE 4');
    if (ftReserve < netOut + feeAmt) _issueFtToSelf(ftReserve, netOut + feeAmt, config);
}
```

And then run the tests `forge test --mt testSellFt -vv`:

Recomendation:

```
if (tokenOut == ft) {
    uint256 ftReserve = getTransientFtReserve();
    console.log("Reserves", ftReserve);
    console.log("Real Balance", ft.balanceOf(address(this)));
    console.log("HERE 4");
    if (ftReserve < netOut + feeAmt) _issueFtToSelf(ftReserve, netOut + feeAmt, config);
}
market.mint(address(this), debtTokenAmtIn);
```

Term Structure: The purpose of the temporary ft reserve is to calculate the change in ft before and after the transaction. The change in ft occurs after the calculation and will not break the contract.

This may cause the debt to increase. I think this is just a low-level problem, because ft can be used to repay the debt, and the debt is unchanged in general.

Term Structure: Addressed in PR 4 and PR 5.

Cantina Managed: Fixes verified.

3.1.4 `_badDebtMapping[collateral]` can be unintentionally rewrite

Submitted by BengalCatBalu

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Summary: Let's consider the function `redeemFromMarket`:

```
function _redeemFromMarket(address order, OrderInfo memory orderInfo) internal returns (uint256 totalRedeem) {
    uint256 ftReserve = orderInfo.ft.balanceOf(order);
    ITermMaxOrder(order).withdrawAssets(orderInfo.ft, address(this), ftReserve);
    orderInfo.ft.safeIncreaseAllowance(address(orderInfo.market), ftReserve);
    totalRedeem = orderInfo.market.redeem(ftReserve, address(this));
    if (totalRedeem < ftReserve) {
        // storage bad debt
        (,,, address collateral,) = orderInfo.market.tokens();
        _badDebtMapping[collateral] = ftReserve - totalRedeem;
    }
    emit RedeemOrder(msg.sender, order, ftReserve.toUint128(), totalRedeem.toUint128());

    delete _orderMapping[order];
    _supplyQueue.remove(_supplyQueue.indexOf(order));
    _withdrawQueue.remove(_withdrawQueue.indexOf(order));
}
```

The key thing we can notice here is that some execution of this function may result in the `_badDebtMapping` variable being overwritten. The key error here is that = is used instead of +=.

All orders that are created using the protocol are bound to some market. Market is a set of collateral, `debtToken`, ft, xt, gt tokens. However, obviously, different marts can have the same collateral if the `debtToken` is different.

Then, obviously, if this function is executed for two two orders with marquees that have the same collateral (maybe they will be orders of the same marquee) - in this case, obviously, the second transaction will overwrite the variable storing.

Let's see how this affects users. There is a function `dealBadDebt` - which just withdraws the funds of bad debts.

```
function dealBadDebt(address recipient, address collaretal, uint256 amount)
    external
    onlyProxy
    returns (uint256 collateralOut)
{
    _accruedInterest();
    uint256 badDebtAmt = _badDebtMapping[collaretal];
    if (badDebtAmt == 0) revert NoBadDebt(collaretal);
    if (amount > badDebtAmt) revert InsufficientFunds(badDebtAmt, amount);
    uint256 collateralBalance = IERC20(collaretal).balanceOf(address(this));
    collateralOut = (amount * collateralBalance) / badDebtAmt;
    IERC20(collaretal).safeTransfer(recipient, collateralOut);
    _badDebtMapping[collaretal] -= amount;
    _accretingPrincipal -= amount;
    _totalFt -= amount;
}
```

And we see that the amount of funds received is directly related to the maximum in badDebtAmt - that is, users lose funds as a result of such overwriting.

Impact Explanation: High - user funds losts.

Likelihood Explanation: I think the probability is also high, since the chances of the collateral being overwritten are very high.

- First of all - collateral is the same for different orders from the same marketplace, as well as for different orders from different marketplaces but with common collateral.
- Secondly, the probability that `totalRedeem < ftReserve` is quite high as there is a withdrawal fee which is likely to make this condition true more often than not.

Recomendation: Change to +=.

Term Structure: Addressed in PR 4 and PR 5.

Cantina Managed: Fixes verified.

3.1.5 Withdrawing from the vault may lead to a loss if the order balances are less than the total amount withdrawn

Submitted by Nyksx

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Finding Description: When the user withdraws, the `withdrawAssets` function calls the _burnFromOrder() if the order is not mature yet.

```
else if (block.timestamp < orderInfo.maturity) {
    // withraw ft and xt from order to burn
    uint256 maxWithdraw = orderInfo.xt.balanceOf(order).min(orderInfo.ft.balanceOf(order));


    if (maxWithdraw < amountLeft) {
        amountLeft -= maxWithdraw;
        _burnFromOrder(ITermMaxOrder(order), orderInfo, maxWithdraw);
        //@audit-issue it burns from the order but didnt transfer
        ++i;
    } else {
        _burnFromOrder(ITermMaxOrder(order), orderInfo, amountLeft);
        asset.safeTransfer(recipient, amountLeft);
        amountLeft = 0;
        break;
    }
} else {
    // ignore orders that are in liquidation window
    ++i;
}
```

The function can only burn an amount equal to the minimum balance of the XT or FT token that the order has. If the withdrawal amount exceeds the maximum burnable amount, the function updates the `amountLeft`, burns the necessary amount from the current order, and then moves on to the next order to fulfill the withdrawal.

The problem is that the function updates the `amountLeft` but does not transfer the `maxWithdraw` amount of tokens to the user after burning from the order. When it moves on to the next order, it only transfers `amountLeft - maxWithdraw` to the user.

Impact Explanation: If the balance of orders XT or FT is less than the user's withdrawal amount, the user will incur a loss of funds.

Proof of Concept:

```
function testRedeemWhenTheXtBalanceLessThanWithdrawalAmount() public {

    vm.warp(currentTime + 3 days);
    address lper2 = vm.randomAddress();
    uint256 amount2 = 10000e8;
    res.debt.mint(lper2, amount2);
    vm.startPrank(lper2);
    res.debt.approve(address(vault), amount2);
    uint256 shares = vault.deposit(amount2, lper2);
    vm.stopPrank();

    vm.startPrank(curator);
    address order2 = address(vault.createOrder(market2, maxCapacity, 0, orderConfig.curveCuts));
    uint256[] memory indexes = new uint256[](2);
    indexes[0] = 1;
    indexes[1] = 0;
    vault.updateSupplyQueue(indexes);

    res.debt.mint(curator, 10000e8);
    res.debt.approve(address(vault), 10000e8);
    vault.deposit(10000e8, curator);

    vm.stopPrank();

    vm.warp(currentTime + 4 days);


    // Buy some XT so the XT balance will be less than the withdrawal amount
    {
        address taker = vm.randomAddress();
        uint128 tokenAmtIn = 1000e8;
        res.debt.mint(taker, tokenAmtIn);
        vm.startPrank(taker);
```

```
    vm.stopPrank();

    address user2 = vm.randomAddress();
    res.debt.mint(user2, amount2*2);
    vm.startPrank(user2);
    res.debt.approve(address(vault), amount2*2);
    uint256 user2Shares = vault.deposit(amount2, user2);
    vm.stopPrank();

    vm.startPrank(curator);
    address order2 = address(vault.createOrder(market2, maxCapacity, 0, orderConfig.curveCuts));
    uint256[] memory indexes = new uint256[](2);
    indexes[0] = 1;
    indexes[1] = 0;
    vault.updateSupplyQueue(indexes);

    res.debt.mint(curator, 10000e8);
    res.debt.approve(address(vault), 10000e8);
    vault.deposit(10000e8, curator);

    vm.stopPrank();

    vm.warp(currentTime + 4 days);
    {
        address taker = vm.randomAddress();
        uint128 tokenAmtIn = 50e8;
        res.debt.mint(taker, tokenAmtIn);
        vm.startPrank(taker);
        res.debt.approve(address(res.order), tokenAmtIn);
        res.order.swapExactTokenToToken(res.debt, res.xt, taker, tokenAmtIn, 1000e8);
        vm.stopPrank();
    }

    vm.warp(currentTime + 40 days);

    {
        address taker = vm.randomAddress();
        uint128 tokenAmtIn = 50e8;
        res.debt.mint(taker, tokenAmtIn);
        vm.startPrank(taker);
        res.debt.approve(address(res.order), tokenAmtIn);
        res.order.swapExactTokenToToken(res.debt, res.xt, taker, tokenAmtIn, 1000e8);
        vm.stopPrank();
    }

    vm.warp(currentTime + 92 days);

    // Order is matured and redeemed
    // User1 withdraws
    vm.prank(user1);
    uint256 user1Redeem = vault.redeem(user1Shares, user1, user1);

    // Curator takes his fees
    uint256 performanceFee = vault.performanceFee();
    vm.prank(curator);
    vault.withdrawPerformanceFee(curator, performanceFee);

    // User2 withdraws
    vm.prank(user2);
    uint256 user2Redeem = vault.redeem(user2Shares, user2, user2);

    console.log("user1 shares:", user1Shares);
    console.log("user2 shares:", user2Shares);
    console.log("user1 asset amount:", user1Redeem);
    console.log("user2 asset amount:", user2Redeem);

}
```

Even if two users deposit the same amount at the same time, one of them can withdraw more tokens simply because they withdraw before the curator takes performance fees. As performance fees increase, the difference can become even greater.

```
Logs:
  user1 shares: 99994520848
  user2 shares: 99994520848
  user1 asset amount: 100671216158
  user2 asset amount: 100318322561
```

Recommendation: Exclude the performance fees from the `totalAssets()`.

Term Structure: Addressed in PR 4 and PR 5.

Cantina Managed: Fixes verified.

3.2.8 Redeem fee is incorrectly factored into bad debt mapping due to incorrect logic

Submitted by 0xJoos, also found by BengalCatBalu

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Summary: Redeem fee is incorrectly factored into bad debt mapping due to incorrect logic.

Finding Description: When redeeming an order from the `TermMaxVault` contract, the flow is as follows:

- `TermMaxVault::redeemOrder` is called, which will delegatecall to `OrderManager::redeemOrder` → `OrderManager::_redeemFromMarket`, which then calls `TermMaxMarket::redeem`.
- `TermMaxMarket::_redeem`:

```
function _redeem(address caller, address recipient, uint256 ftAmount) internal returns (uint256
↪  debtTokenAmt) {
    //
    debtTokenAmt += ((debtToken.balanceOf(address(this))) * proportion) / Constants.DECIMAL_BASE_SQ;
    uint256 feeAmt;
    if (mConfig.feeConfig.redeemFeeRatio > 0) { // <<<
        feeAmt = (debtTokenAmt * mConfig.feeConfig.redeemFeeRatio) / Constants.DECIMAL_BASE; // <<<
        debtToken.safeTransfer(mConfig.treasurer, feeAmt); // <<<
        debtTokenAmt -= feeAmt; // <<<
    } // <<<
    debtToken.safeTransfer(recipient, debtTokenAmt);
    emit Redeem(
        caller, recipient, proportion.toUint128(), debtTokenAmt.toUint128(), feeAmt.toUint128(),
        ↪  deliveryData
    );
}
```

- `OrderManager::_redeemFromMarket`:

```
function _redeemFromMarket(address order, OrderInfo memory orderInfo) internal returns (uint256
↪  totalRedeem) {
    uint256 ftReserve = orderInfo.ft.balanceOf(order);
    ITermMaxOrder(order).withdrawAssets(orderInfo.ft, address(this), ftReserve);
    orderInfo.ft.safeIncreaseAllowance(address(orderInfo.market), ftReserve);
    totalRedeem = orderInfo.market.redeem(ftReserve, address(this));
    if (totalRedeem < ftReserve) { // <<<
        (,,, address collateral,) = orderInfo.market.tokens(); // <<<
        _badDebtMapping[collateral] = ftReserve - totalRedeem; // <<<
    } // <<<
    emit RedeemOrder(msg.sender, order, ftReserve.toUint128(), totalRedeem.toUint128());

    delete _orderMapping[order];
    _supplyQueue.remove(_supplyQueue.indexOf(order));
    _withdrawQueue.remove(_withdrawQueue.indexOf(order));
}
```

- `OrderManager::dealBadDebt`:

```
function dealBadDebt(address recipient, address collaretal, uint256 amount)
    external
    onlyProxy
    returns (uint256 collateralOut)
{
    _accruedInterest();
    uint256 badDebtAmt = _badDebtMapping[collaretal]; // <<<
    if (badDebtAmt == 0) revert NoBadDebt(collaretal);
    if (amount > badDebtAmt) revert InsufficientFunds(badDebtAmt, amount);
    uint256 collateralBalance = IERC20(collaretal).balanceOf(address(this));
    collateralOut = (amount * collateralBalance) / badDebtAmt; // <<<
    IERC20(collaretal).safeTransfer(recipient, collateralOut);
    _badDebtMapping[collaretal] -= amount;
    _accretingPrincipal -= amount;
    _totalFt -= amount;
}
```

Focusing on the lines marked with `// <<<`, when there is a `redeemFeeRatio` set for the market, `debtTokenAmt` will be reduced by the `feeAmt`. This `debtTokenAmt` is then returned to `_redeemFromMarket` as `totalRedeem`. Since `market.redeem` is passing `ftReserve` in the params, if there is a redeem fee set in the market, `totalRedeem` will always be less than `ftReserve`.

This will result in the deduction of the `debtTokenAmt` due to the collection of the redeem fee to be factored into the bad debt mapping, causing the bad debt to be inflated. Subsequently calling `dealBadDebt` will cause the collateral to be transferred to the recipient to be diluted due to `badDebtAmt` being inflated from the aforementioned issue.

Proof of Concept: This is a simple proof of concept where the curator calls `vault.redeemOrder`, and does not expect any change in the bad debt mapping. However, due to an incorrect logic, the redeem fee collected increases the bad debt in the collateral.

Edit the `redeemFeeRatio` in `testdata.json` to `"redeemFeeRatio": "1000000"`. Insert the following in `Vault.t.sol`.

```
function test_RedeemFeeContributingToBadDebt() public {
    // Set up vault
    vm.warp(currentTime + 2 days);
    buyXt(48.219178e8, 1000e8);

    vm.warp(currentTime + 3 days);
    address lper2 = vm.randomAddress();
    uint256 amount2 = 10000e8;
    res.debt.mint(lper2, amount2);
    vm.startPrank(lper2);
    res.debt.approve(address(vault), amount2);
    vault.deposit(amount2, lper2);
    vm.stopPrank();

    // Curator creates orders in market
    vm.startPrank(curator);
    address order2 = address(vault.createOrder(market2, maxCapacity, 0, orderConfig.curveCuts));
    uint256[] memory indexes = new uint256[](2);
    indexes[0] = 1;
    indexes[1] = 0;
    vault.updateSupplyQueue(indexes);
    // curator deposits debt tokens into vault
    res.debt.mint(curator, 10000e8);
    res.debt.approve(address(vault), 10000e8);
    vault.deposit(10000e8, curator);

    vm.stopPrank();

    (, IERC20 xt,,,) = market2.tokens();
    vm.warp(currentTime + 92 days);
    {
        address taker = vm.randomAddress();
        uint128 tokenAmtIn = 50e8;
        res.debt.mint(taker, tokenAmtIn);
        vm.startPrank(taker);
        res.debt.approve(address(order2), tokenAmtIn);
        ITermMaxOrder(order2).swapExactTokenToToken(res.debt, xt, taker, tokenAmtIn, 1000e8);
        vm.stopPrank();
    }
```